
Курс: Рекомендательные системы

Занятие 6:

Масштабирование

Лашинин Олег
18.02.25

План занятия

- Оффлайн и онлайн рекомендации
- Верхнеуровневый ml-дизайн системы
- Recent Inference
- Строим двухуровневую модель через подход «datamart» + шардирование + параллелизация
- Строим двухуровневую модель через Профиль пользователя + шардирование + параллелизация
- Масштабирование по айтемам (EASE и компания)
- Масштабирование по юзерам (некоторые приемы)
- Масштабирование по данным

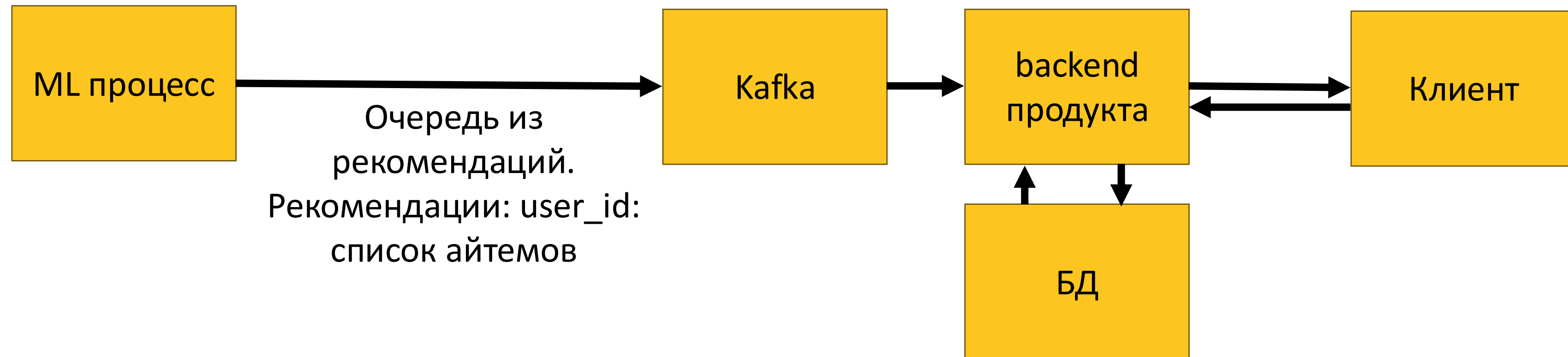
Почему сегодня эта тема?

Чем больше юзеров, тем больше импакт от ваших действий с рек. системой, но тем сложнее что-то делать

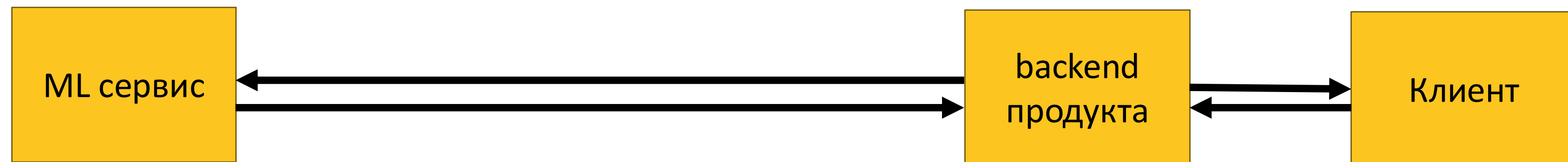
Онлайн и оффлайн рекомендации

- Есть синхронный/асинхронный формат отдачи рекомендаций в backend
- Есть возможность предусчитывать рекомендации оффлайн или делать это в онлайн
- Есть время задержки учета событий пользователя. Это время между действием юзера и моментом, когда оно учлось

Синхронный и асинхронный форматы



Backend асинхронно вычитывает очередь, сохраняет к себе и по запросу выдает сохраненные списки



В синхронном формате backend ходит в ml service и запрашивает генерацию списков рекомендаций в моменте

Предрасчитанные списки и генерация «на лету»

Предрасчитанный список:

User_id 1: item_ids: [1, 2, 3, 4, 5, 6, 7]

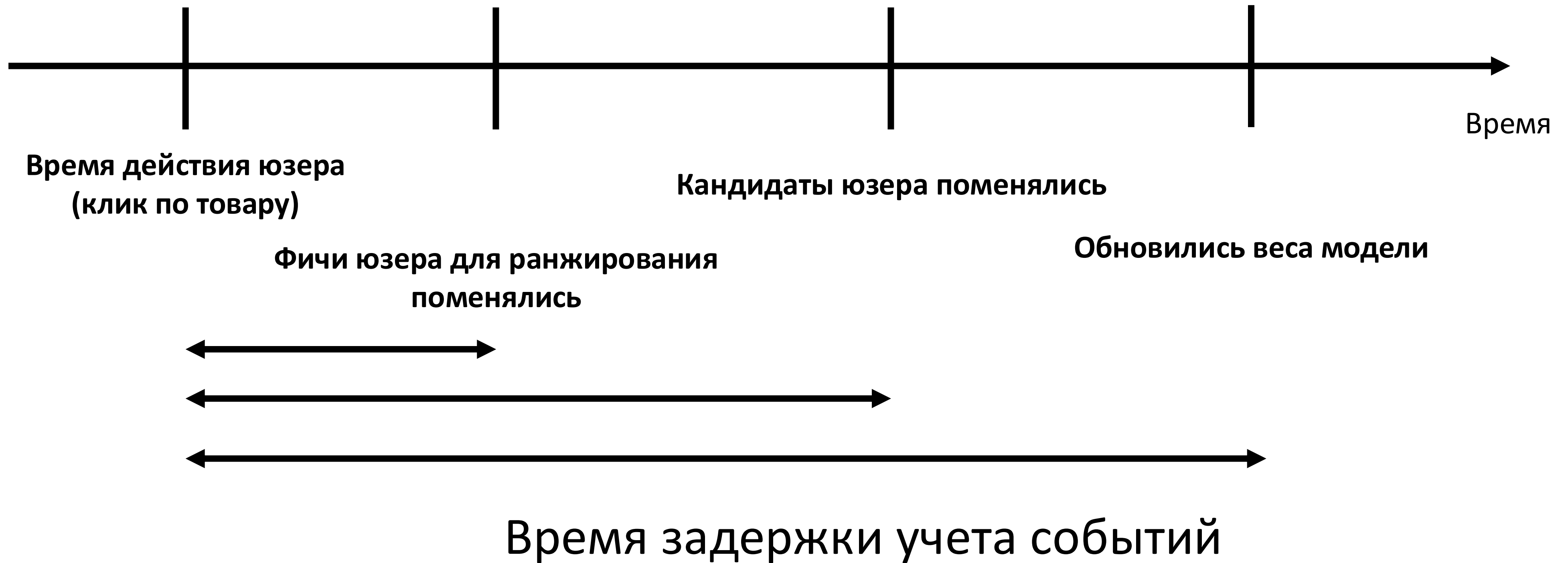
Расчет списка на лету по user_id 1:

- собери данные по юзеру
- {сгенерируй кандидатов}
- {собери данные по кандидатам}
- {сделай ранжирование кандидатов по всем фичам}
- {сделай бизнес логику}
- верни итоговый список

Или:

- возьми вектор юзера
- сделай ANN по айтемам
- верни ближайшие айтемы

Время реагирования на действия



Характерные размеры: 5 секунд, 1 минута, 1 час, 1 день, 1 неделя

Онлайн и оффлайн рекомендации

Контракт с бекендом	Способ расчета рекомендаций	Задержка учета событий	Вердикт онлайн/оффлайн
асинхронный	предрасчитанные	1 день	оффлайн
асинхронный	предрасчитанные	1 секунда	???
асинхронный	в моменте	1 день	оффлайн
асинхронный	в моменте	1 секунда	онлайн
синхронный	предрасчитанные	1 день	оффлайн
синхронный	предрасчитанные	1 секунда	???
синхронный	в моменте	1 день	Оффлайн????
синхронный	в моменте	1 секунда	онлайн

Про скорость обновления рекомендаций

- Чем быстрее мы учтем действие клиента, тем лучше
- Если последние действия не важны, система не должна их учитывать
- Если важны, качество должно вырасти

Верхнеуровневый дизайн рек. системы

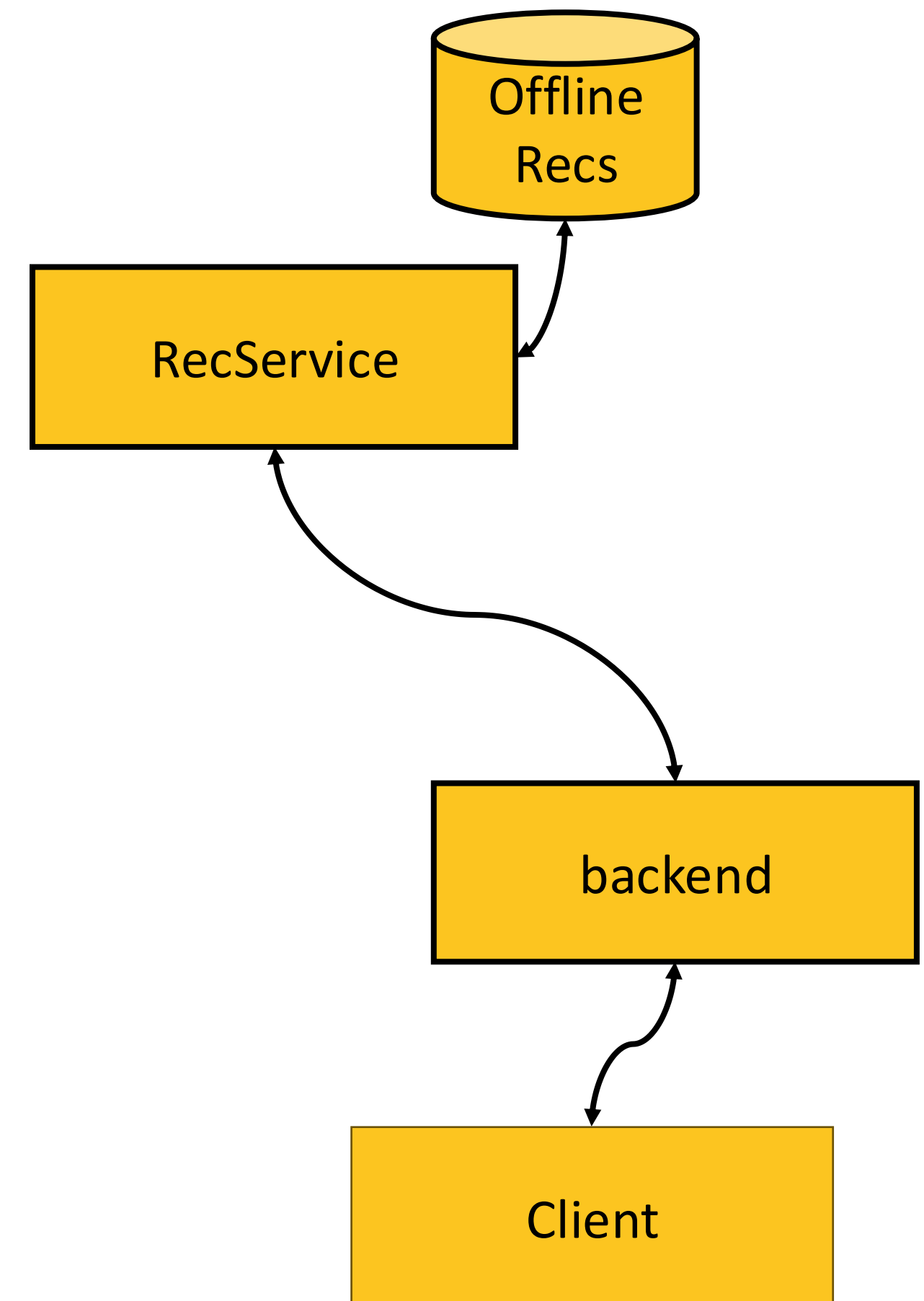
Есть БД с рекомендациями по клиенту

Есть сервис, который умеет ходить в БД и по запросу возвращать рекомендации

На любой запрос мы можем выдать рекомендации

Если клиента нет в БД, надо сделать заглушку («фоллбек»)

Это базовый старт



Верхнеуровневый дизайн рек. системы

Можно сделать базу векторов юзеров и айтемов и делать ANN

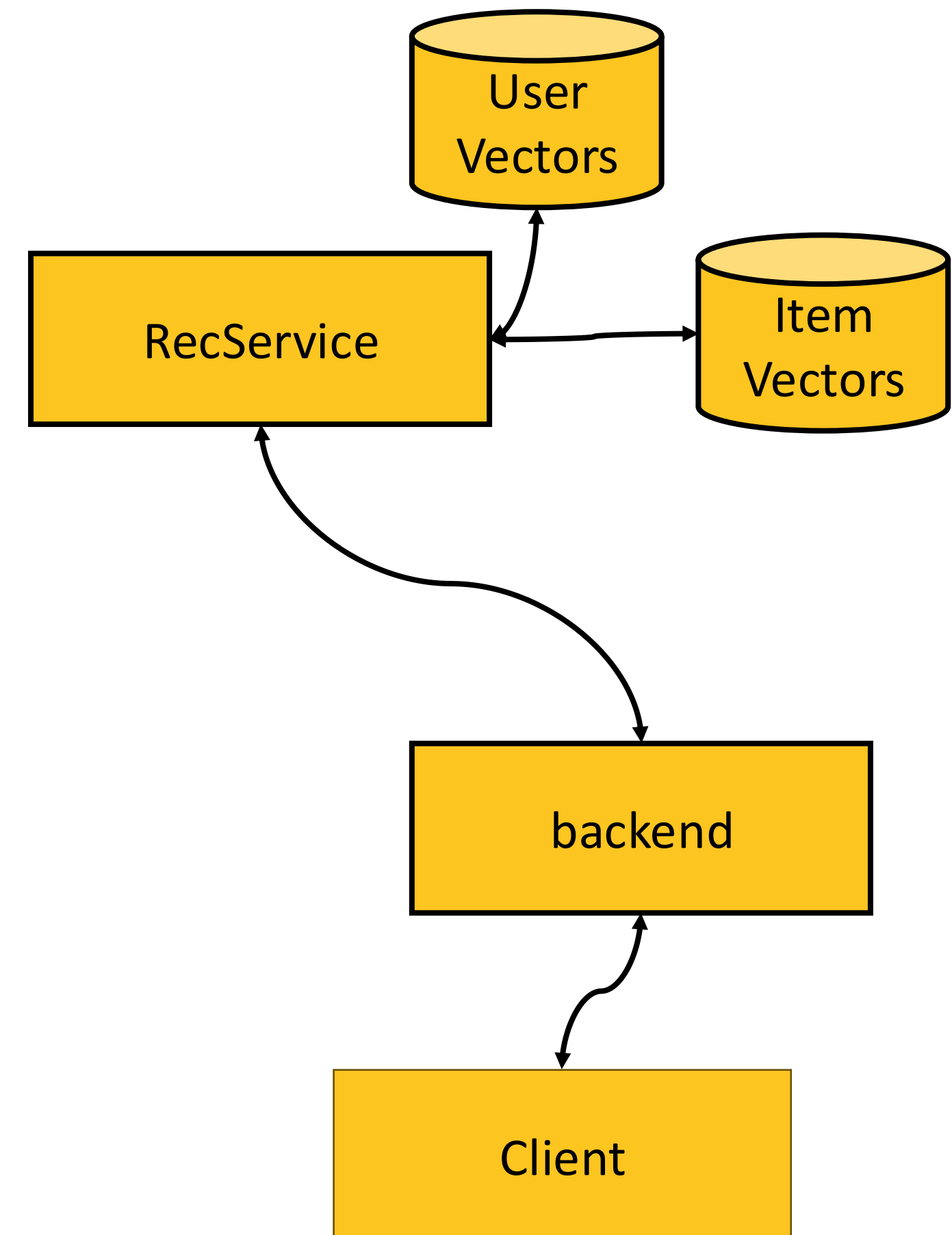
Тогда мы не храним предрасчитанные рекомендации.

Плюсы:

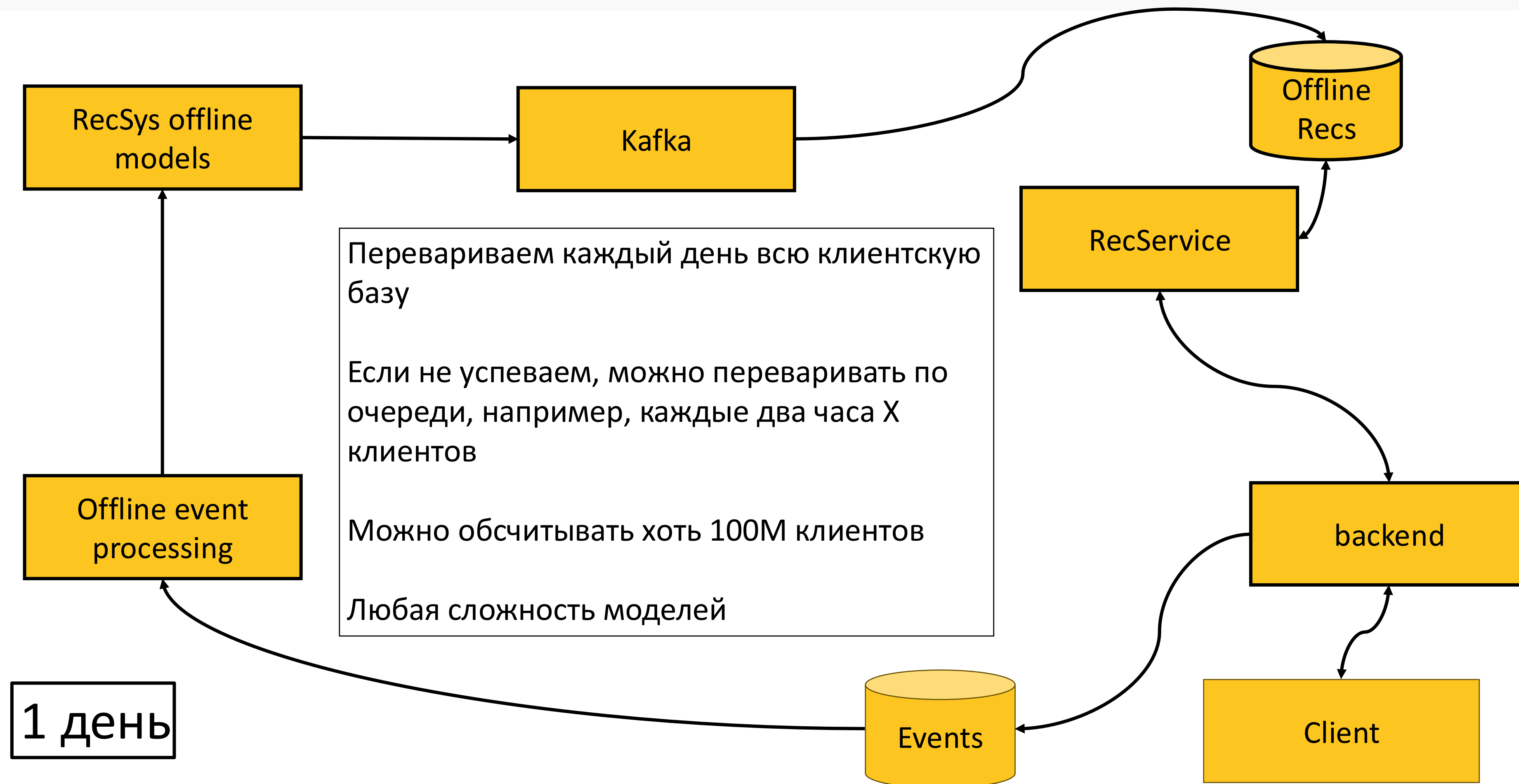
- меньше записей в БД
- можем получать списки рекомендаций любой длины

Минусы:

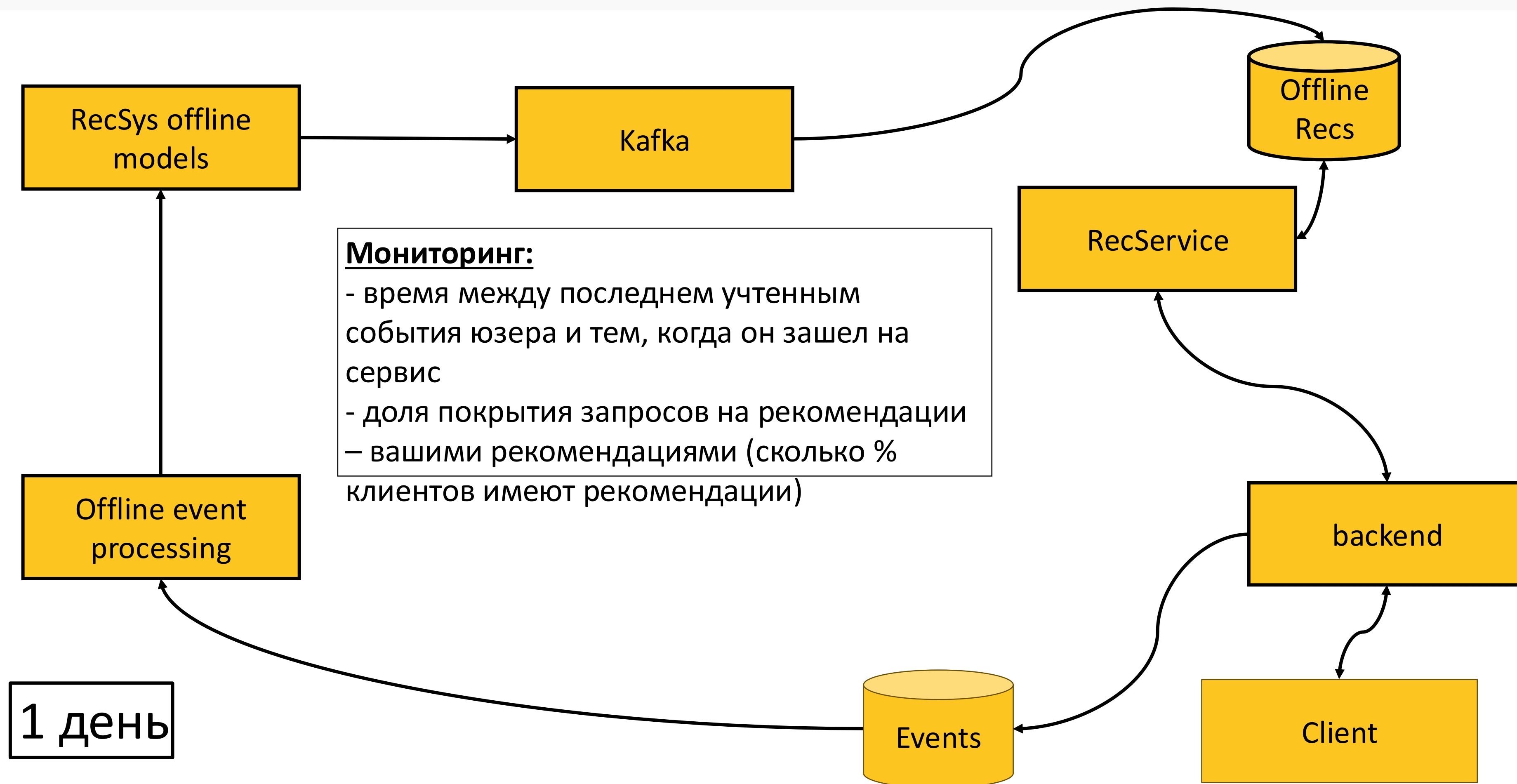
- сложнее реализовать
- менее гибко (нужно, чтобы скор был произведение векторов). В варианте с предрасчетом может быть что угодно (град. бустинг)



Ежедневный пайплайн оффлайн скоринга



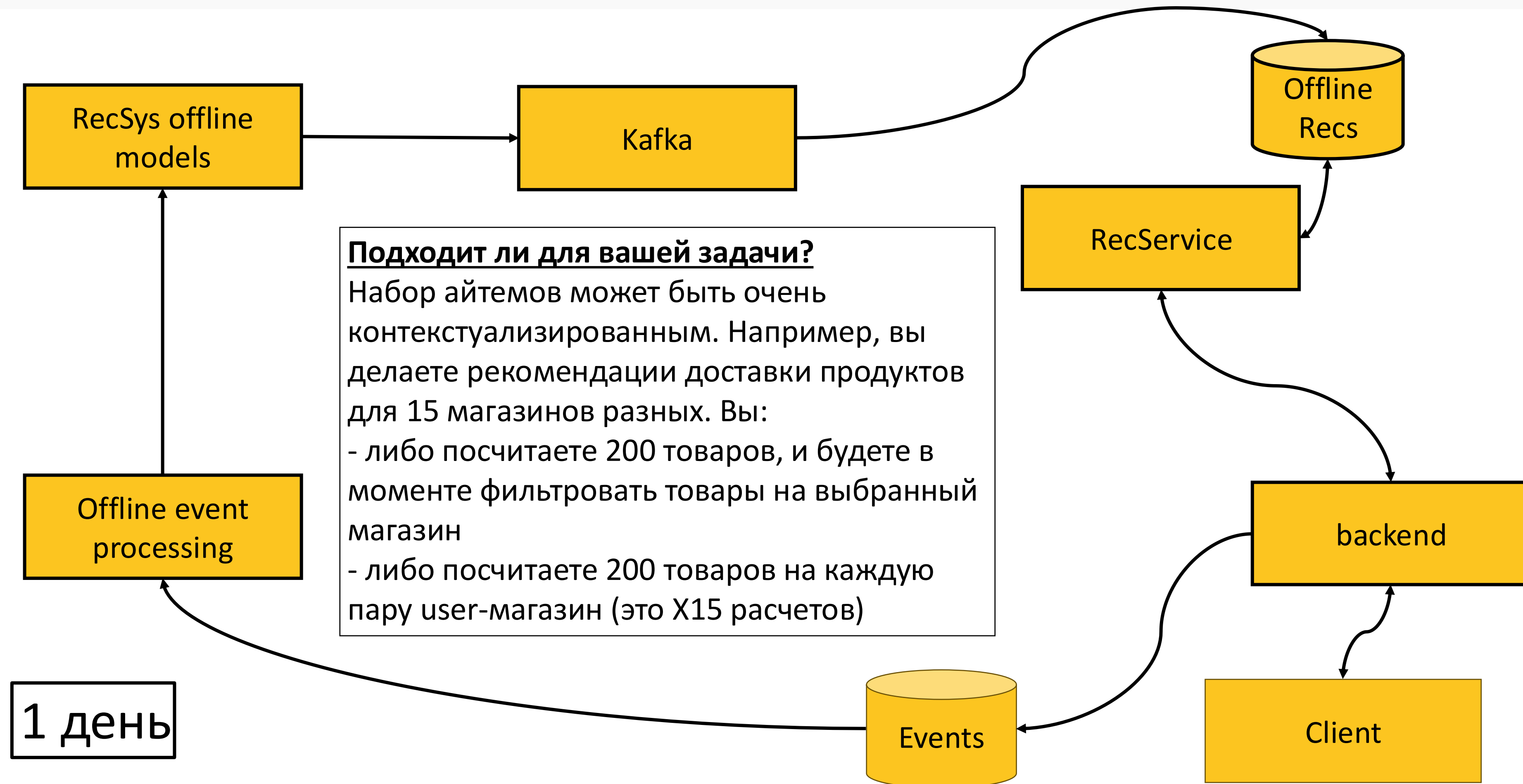
Ежедневный пайплайн оффлайн скоринга



Ежедневный пайплайн оффлайн скоринга



Ежедневный пайплайн оффлайн скоринга



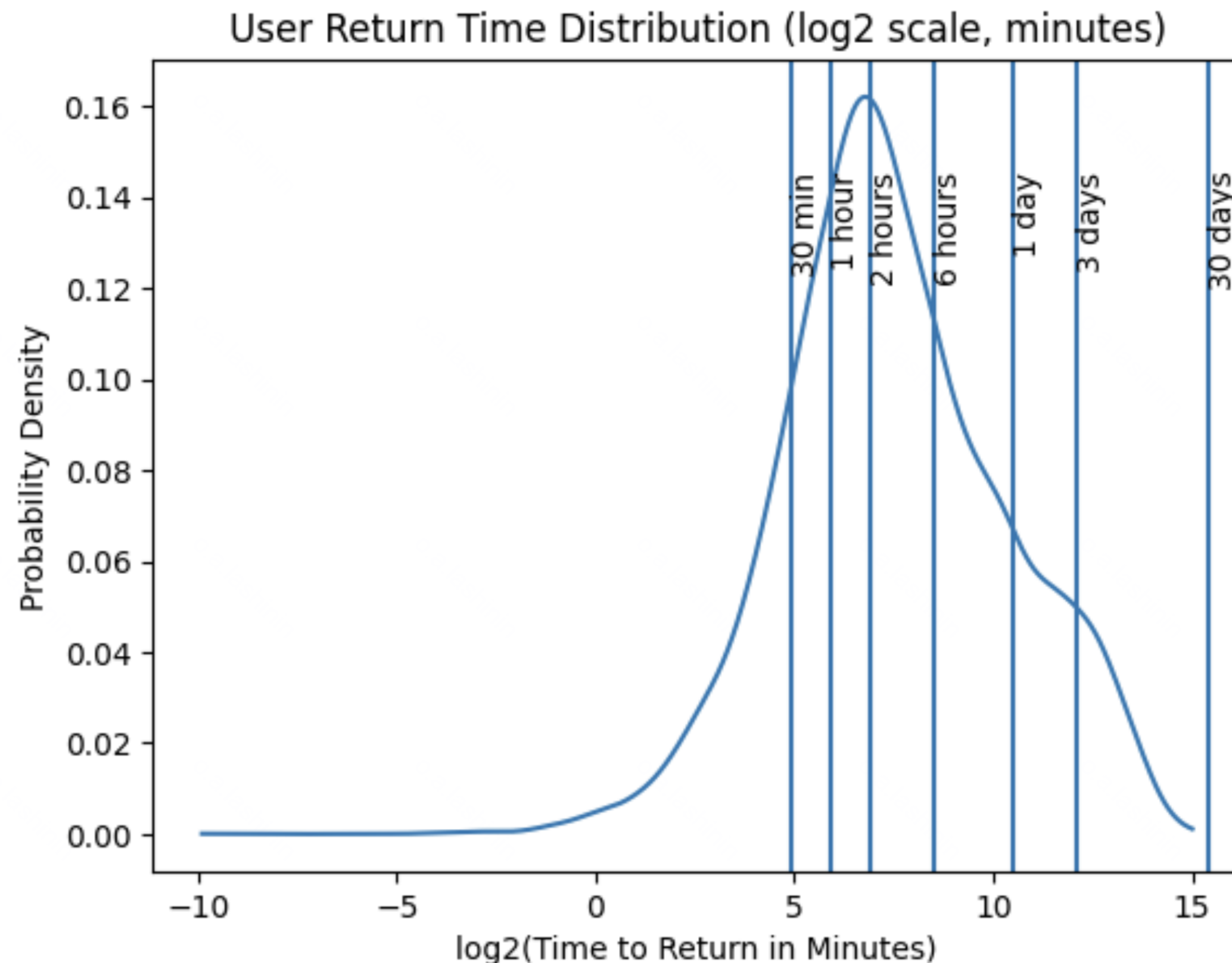
Ежедневный пайплайн оффлайн скоринга



Recent Inference



Recent Inference



У вашего сервиса может быть разное распределение возвращаемости юзеров

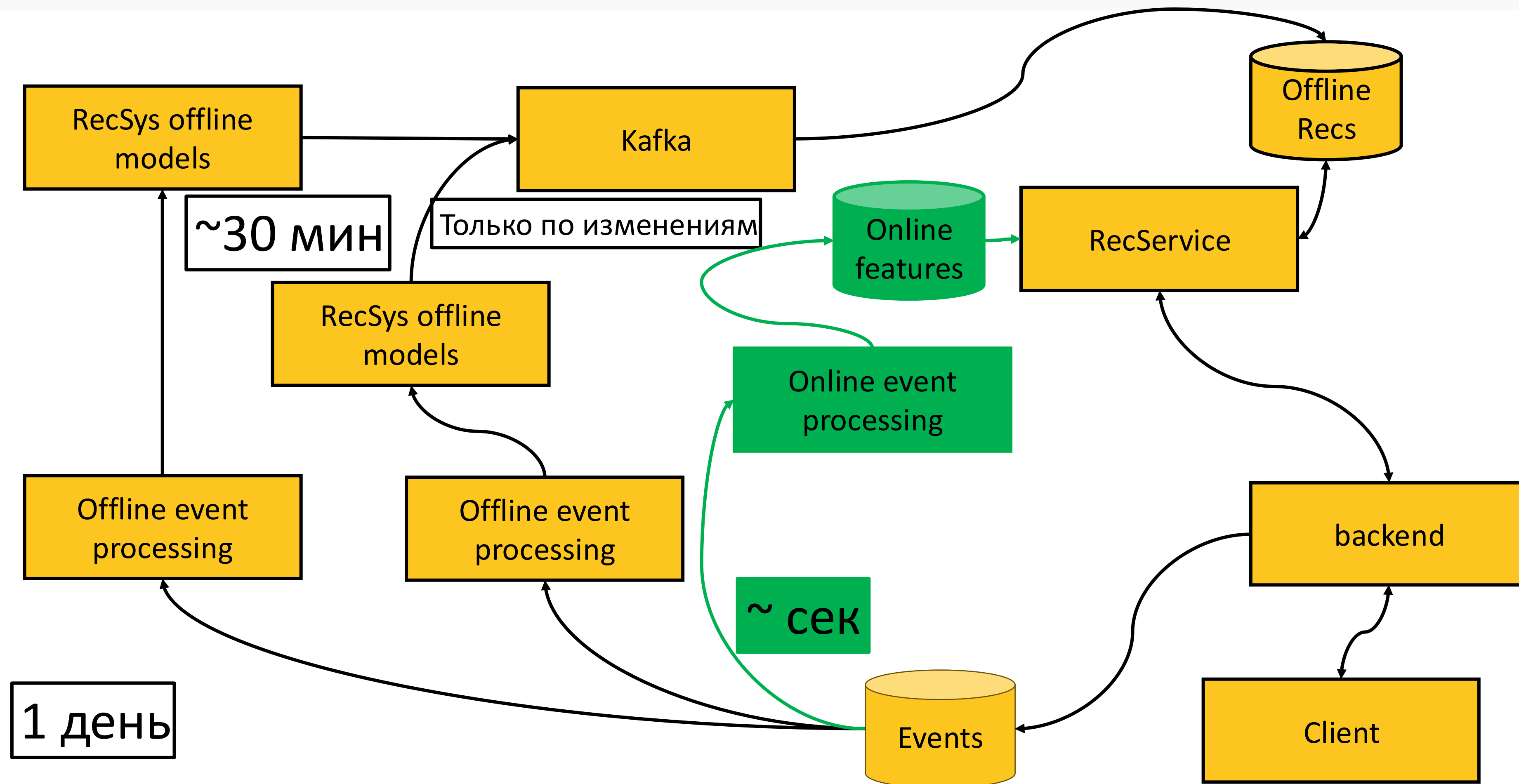
- Если большинство юзеров возвращается в течение 2 часов – recent inference поможет, **время на реагирование уменьшится**

- Если же юзеры возвращаются через 2 дня, то он не улучшит ситуацию

Важно: recent inference – это запаздывающее обновление, в первый заход юзера в день будут **старые** рекомендации, а во второй заход уже новые.

Также триггерное событие на быстрый пересчет – это может быть клик/прослушивание/просмотр, так и показ в ленте (но это x10 событий, которые нагрузят контур recent inference)

Признаки процессим в онлайн (с некоторой задержкой)



Признаки процессим в онлайн (с некоторой задержкой)

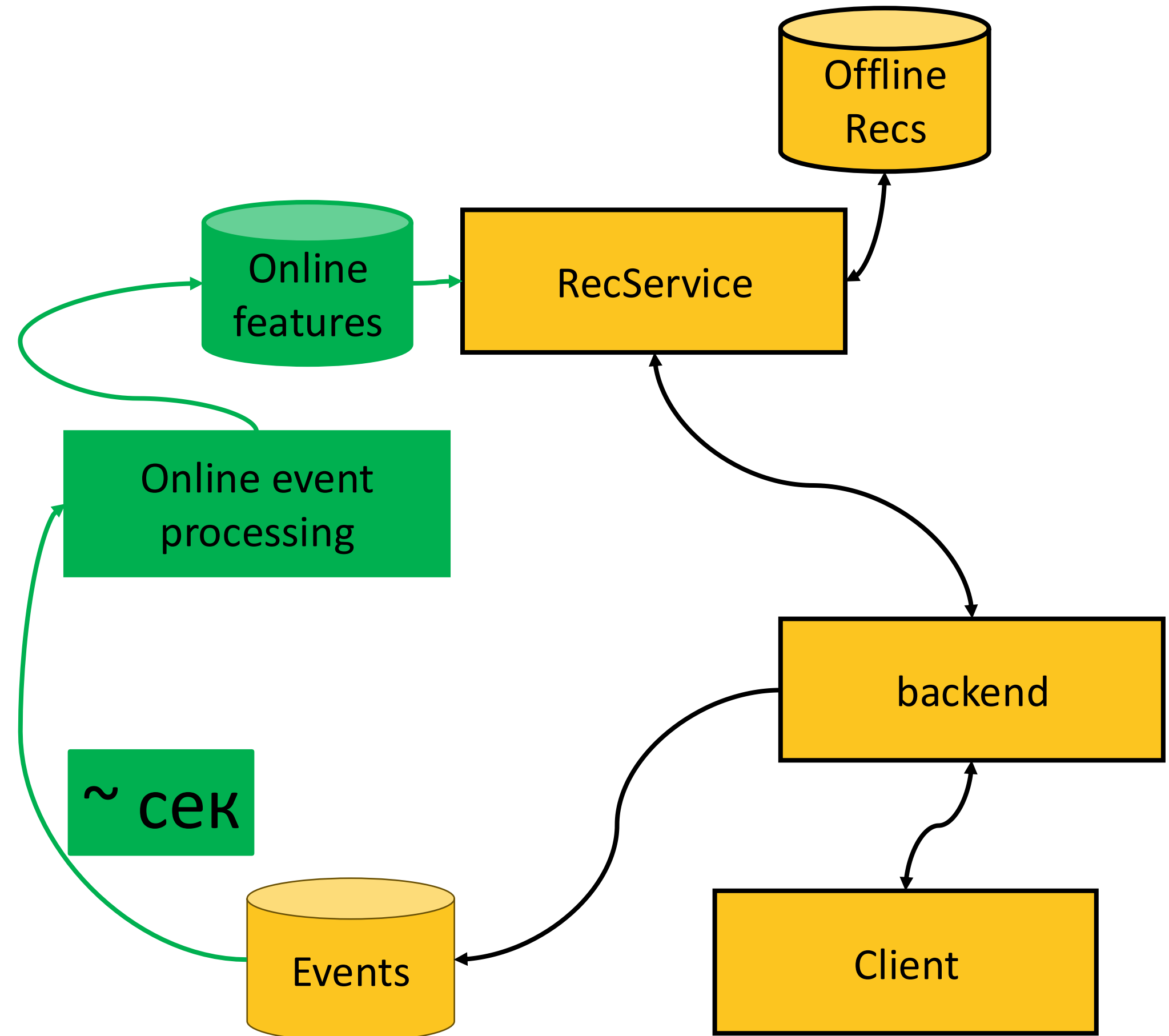
Мы можем обновлять фичи онлайн:

- 1) Для ранжирующей модели
- 2) Для кандидатов

Для ранжирующей модели, например, обновляем фичу:

User-category-num-clicks-from-search-1-day

Онлайн обновление увеличит, например, значение с 0 до 1, и юзеру в этой сессии станет больше показываться товаров из этой категории



Признаки процессим в онлайн (с некоторой задержкой)

Мы можем обновлять фичи онлайн:

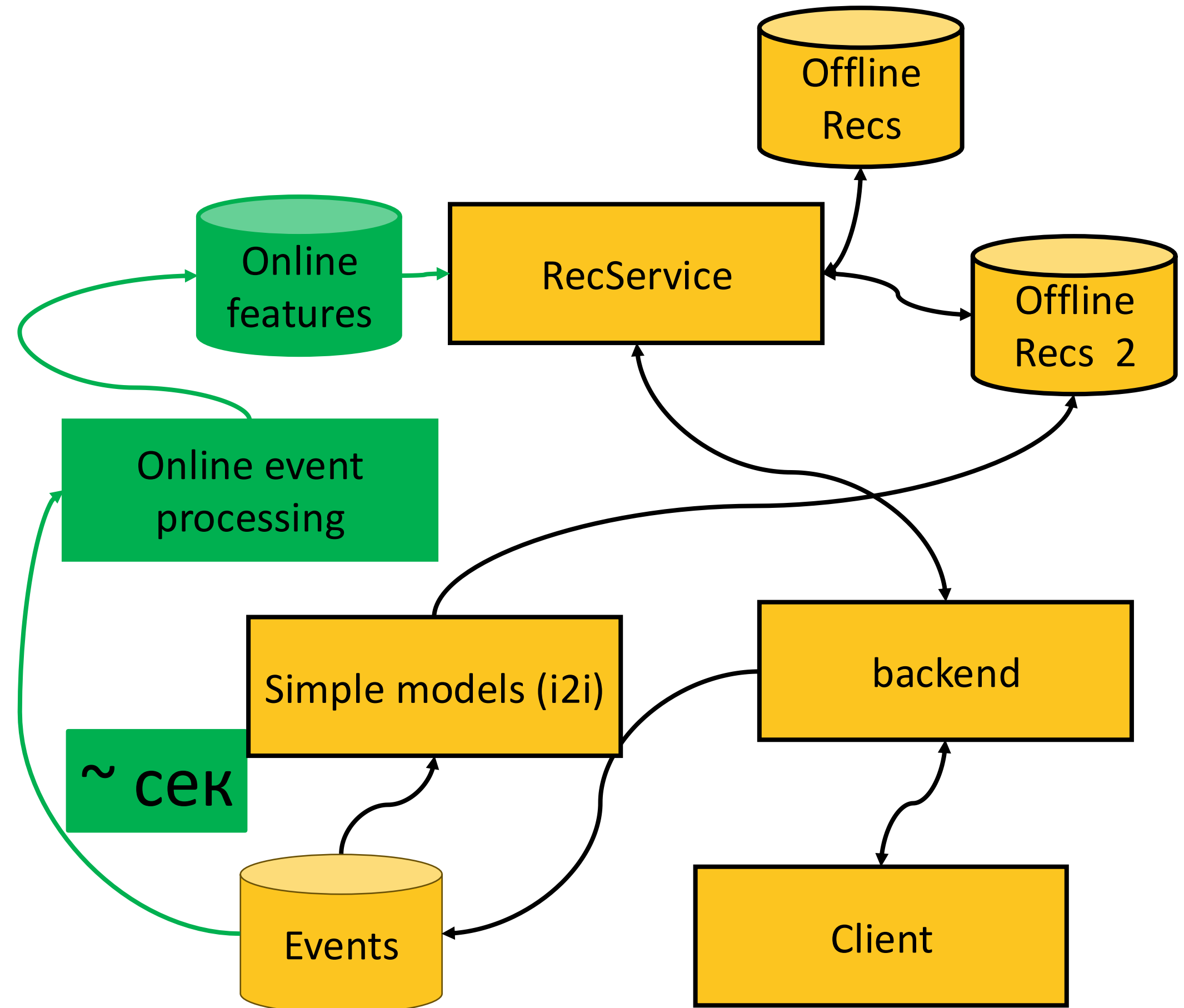
- 1) Для ранжирующей модели
- 2) Для кандидатов

Для ранжирующей модели, например, обновляем фичу:

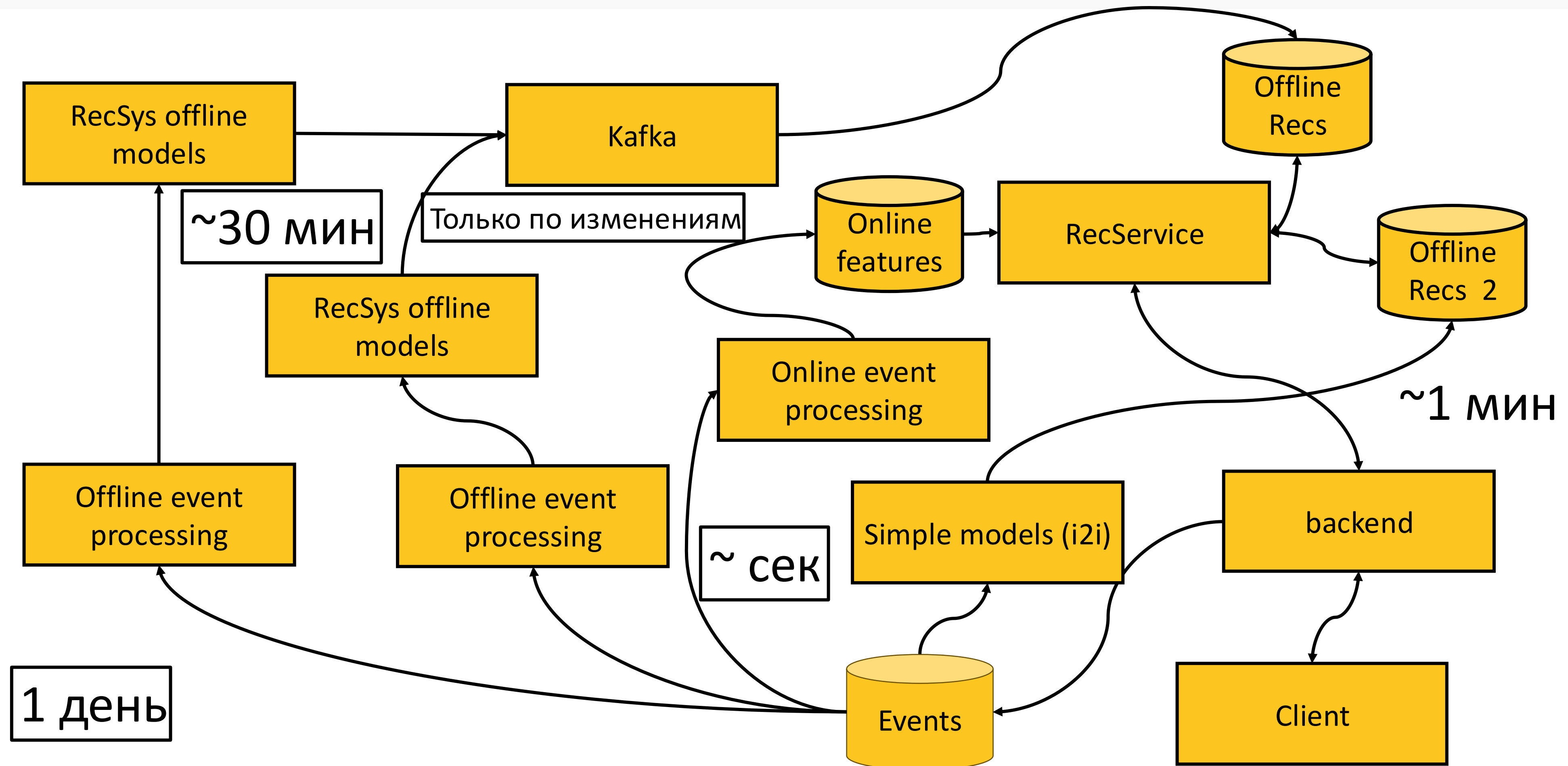
User-category-num-clicks-from-search-1-day

Онлайн обновление увеличит, например, значение с 0 до 1, и юзеру в этой сессии станет больше показываться товаров из этой категории

Для кандидатов мы можем генерировать их на основе последних событий (например, похожие на кликнутые)

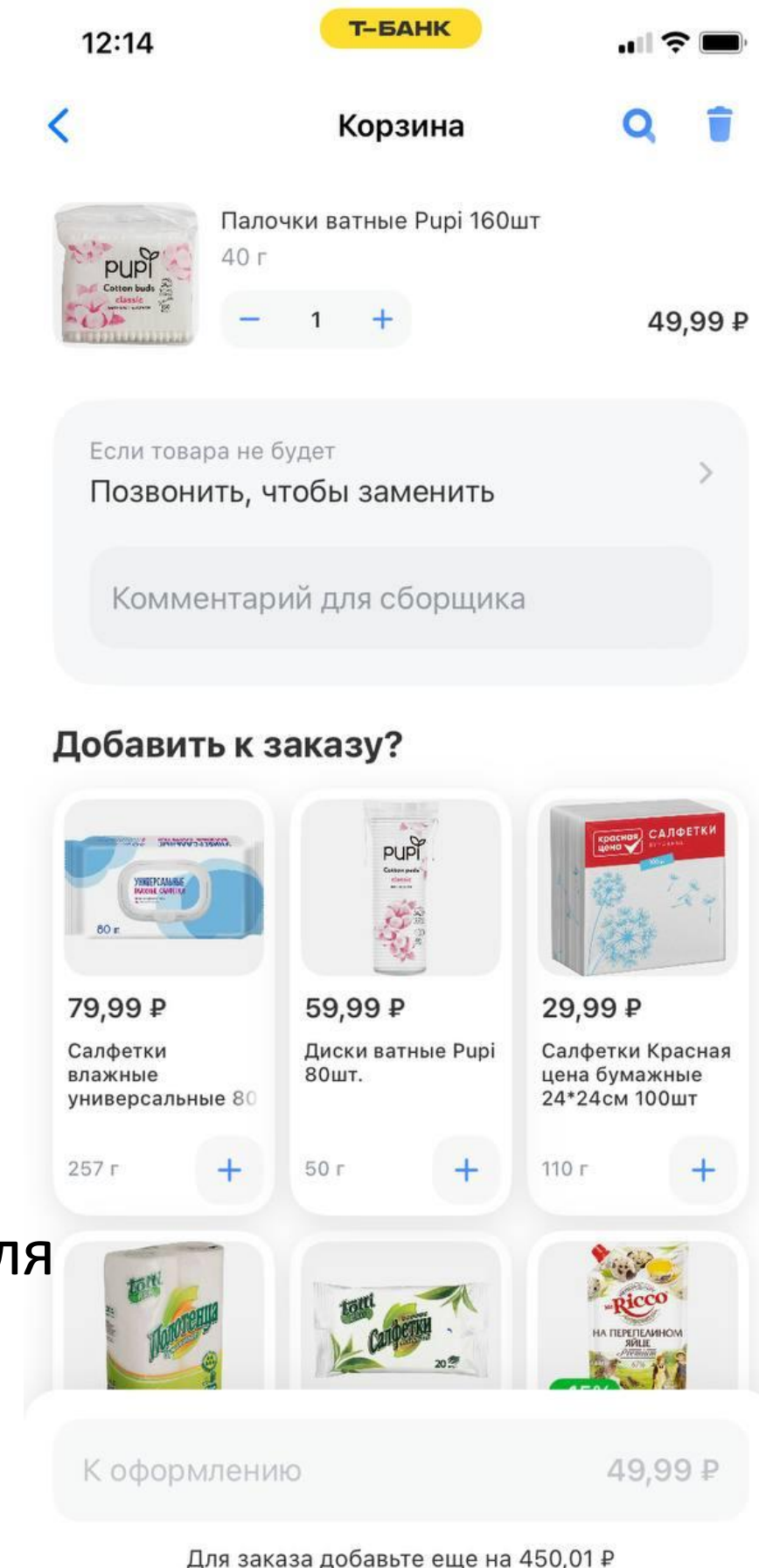
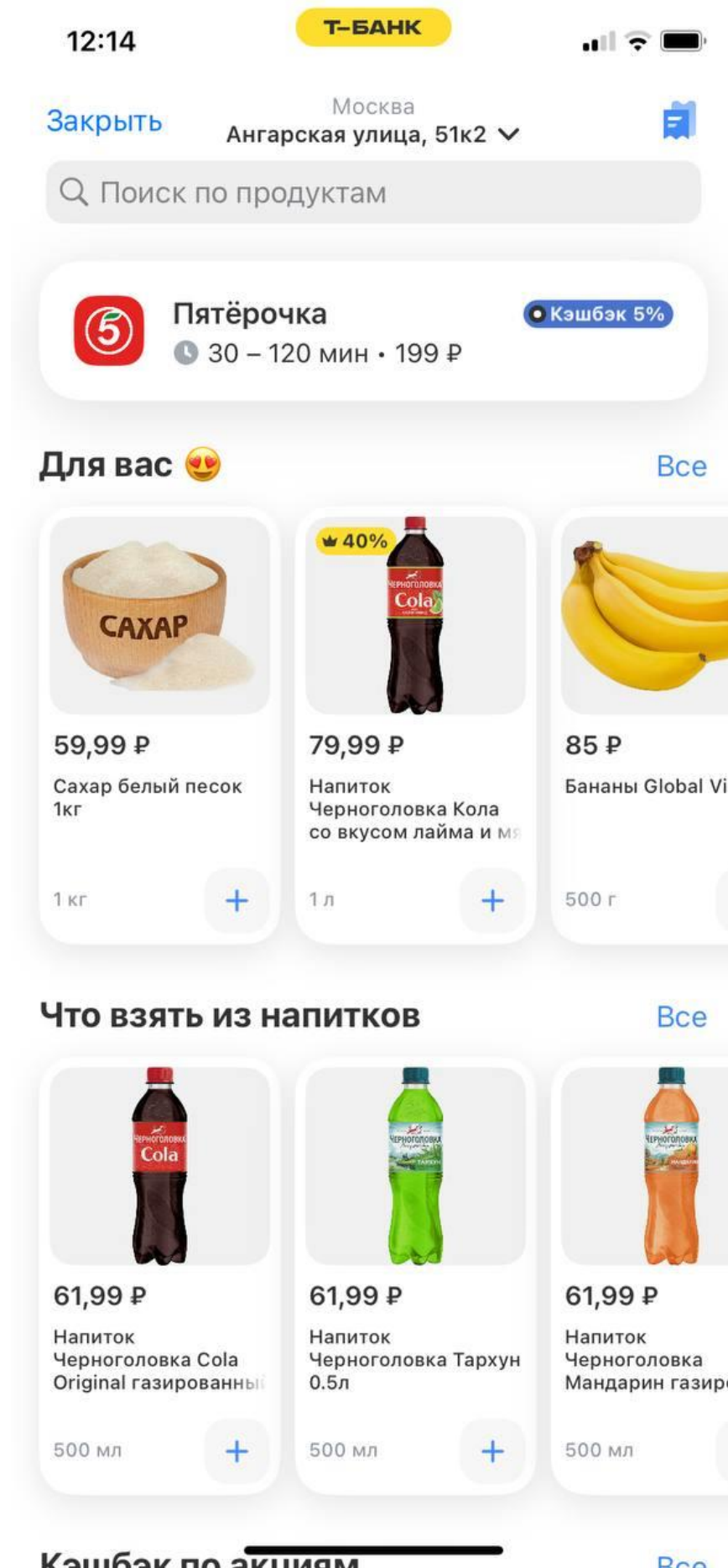


Нужно понимать ценность каждого контура!



Нужно понимать ценность каждого контура!

Вот здесь юзеры могут
возвращаться редко, сделав
заказ,
Recent inference может быть не
нужен (бесполезен)



Вот здесь корзина заранее
неизвестна, нельзя заранее
рассчитывать рекомендации для
каждой корзины, но можно
посчитать это в онлайн

Нужно понимать ценность каждого контура!



Для бизнесаКарьера в АвитоПомощьКаталоги▼#яПомогаю

Вход и регистрацияРазместить объявление

Avito

Все категории

Поиск по объявлениям

Найти

Во всех регионах

СкидкиЖильё для поездкиНовые автоМоллМастера красотыГаражОдежда и обувь

Авто

Недвижимость

Работа

Одежда, обувь, аксессуары

Хобби и отдых

Животные

Готовый бизнес и оборудование

Скидки

Услуги

Электроника

Для дома и дачи

Запчасти

Для детей

Жильё для путешествия

Красота и здоровье

Любовь по отчёту

#яПомогаю

Безграничная Хватамба

Жильё в Мурманске

Зовите всех на Авито

Товар найдётся

Помогайте покупая

Игра Авито и МЧС

Сервисы и услуги Авито

Доставка

Проверка при получении и возможность бесплатно вернуть товар

Автотека

Отчёт с историей авто: пробег, владельцы, сведения о залоге, ДТП и ремонтах

Путешествия

Выбор свободных дат в календаре и оплата онлайн для гарантии заселения

#яПомогаю

Сохраняйте природу и помогайте людям вместе с нами

Рекомендации для вас

Комбинезон женский вечерний

790 Р

Раки живые

99 Р

Москва, Тульская

Ремонт МКПП для Mitsubishi, под ключ

от 6 000 Р за услугу

Санкт-Петербург, р-н Красногвардейский

Арсенальный ангар бескаркасный,...

6 700 000 Р

Самара, Юнгородок

© ООО «КЕХ еКоммерц» 2007–2025. Авито использует рекомендательные технологии.

Правила Авито

Реклама на сайте

О компании

Карьера в Авито

Реклама 0+

БАНК

Бесплатная кредитка

1

Подход к построению offline данных №1 - Datamart

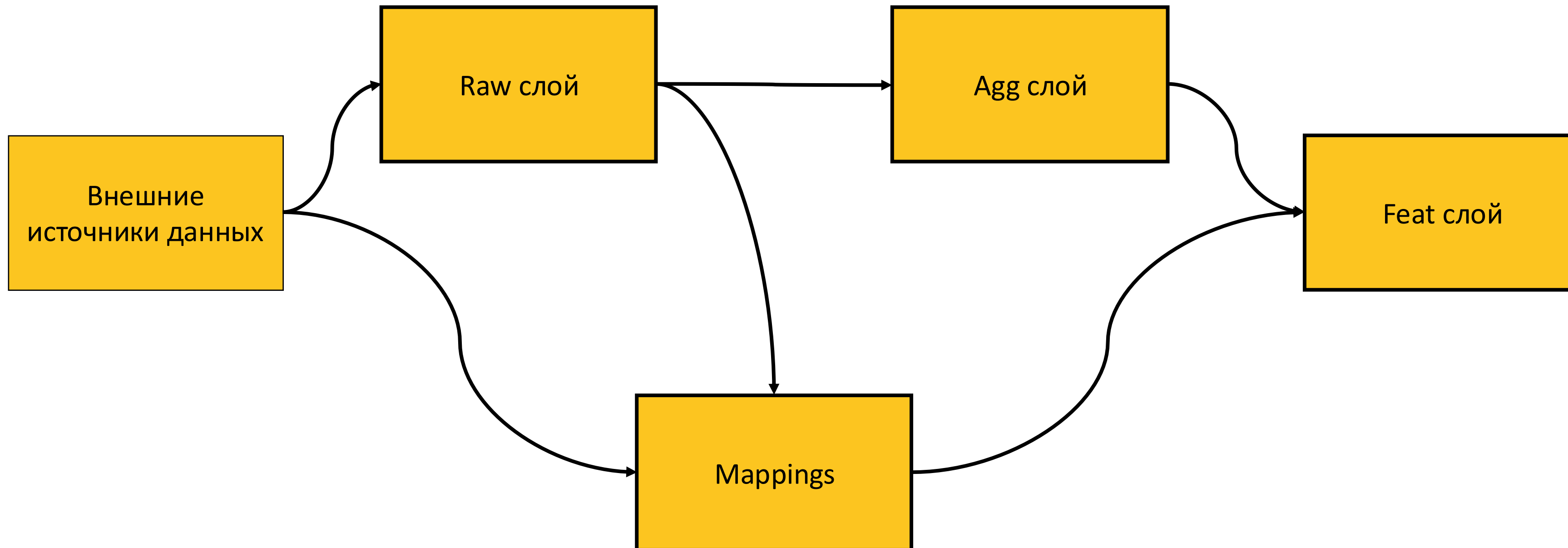


Datamart – подход к систематизации данных, с которыми работают ml-модели в рамках конкретного ml-проекта

Больше всего нужен для градиентного бустинга

Данные в датамарте хранятся за длительный промежуток времени, но собираются инкрементально (это плюс)

Подход к построению offline данных №1 - Datamart



Raw слой



- ▼ • raw
 - catalog
 - goods.pq
 - categories.pq
 - events
 - added_to_cart
 - clicked
 - main_page_open
 - shown
 - 2023-01-01.pq
 - 2023-02-02.pq
 - orders
 - offline
 - online
 - 2023-01-01.pq
 - 2023-02-02.pq

- raw
 - catalog
 - goods.pq
 - categories.pq
 - events
 - added_to_cart
 - clicked
 - main_page_open
 - shown
 - 2023-01-01
 - 001_partition.pq
 - 002_partition.pq
 - 2023-02-02.pq
 - 001_partition.pq
 - 002_partition.pq
 - orders
 - offline
 - online
 - 2023-01-01
 - 001_partition.pq
 - 002_partition.pq
 - 2023-02-02
 - 001_partition.pq
 - 002_partition.pq

Agg слой



- agg
 - events
 - 2023-01-01.pq
 - 2023-01-02.pq
 - orders
 - 2023-01-01.pq
 - 2023-02-02.pq

Client_id	Item_id	Num_showns	Num_clicks	Num_view_from_search

Client_id	Item_id	Num_orders	Avg_price	Total_sum

Feat слой



- feat
 - events
 - user
 - 2023-01-01.pq
 - 2023-02-02.pq
 - item
 - user-item
 - user-category
 - user-author
 - cluster-item
 - cluster-category
 - orders
 - item
 - user-item
 - user-category
 - user-author
 - cluster-item
 - cluster-category

Client_id	Item_id	F_Num_shown s_7d	F_Num_clicks_ 30d	F_Num_view_f rom_search_10 d

Фича:

- посчитана в разрезе определенном
- может быть суммой, средним, максимумом, минимумом, дельтой по времени, отношением (долей) внутри разреза
- может быть посчитана между типами ивентов (конверсия из просмотра в заказ)
- может быть посчитана за разные промежутки времени (день, неделя, год)
- лучше придерживаться одинаковых неймингов, начиная с f_, потом название среза, потом тип фичи, потом суть фичи, потом временное окно
- фичи на дату считаются БЕЗ ДАННЫХ ЗА ЭТУ ДАТУ

Feat слой



- feat
 - events
 - user
 - 2023-01-01.pq
 - 2023-02-02.pq
 - item
 - user-item
 - user-category
 - user-author
 - cluster-item
 - cluster-category
 - orders
 - item
 - user-item
 - user-category
 - user-author
 - cluster-item
 - cluster-category

Client_id	Item_id	F_Num_shown s_7d	F_Num_clicks_ 30d	F_Num_view_f rom_search_10 d

Пример фичи:

F num events good conversion to added to cart from search 30d

F – признак

Num – числовой

Events – посчитан по логам

Good – посчитан в разрезе товаров

Conversion – тип фичи отношение (конверсия)

To_added_to_cart - из просмотра (подразумевается) в добавление в корзину

From_search – только по данным из поиска

30d – за 30 дней

Basis

Базис – формулировка ml задачи, которую должна решать ml модель

Для задачи ранжирования – это обычно табличка вида

- session_id
- timestamp
- user_id
- item_id
- relevance (0, 1, 2, 3...)

Словами – в рамках сессии (session_id) в конкретное время (timestamp) клиент (user_id) провазимодействовал с айтемом (item_id) с разной степенью релевантности (relevance)

Базис строится на основе датамарта (raw слой).

Базис для инференса + трейна строится одинаковым образом! (на обучении нет разметки relevance)

Basis + mappings + features = dataset для обучения и инференса

Общие слова про datamart

На инференсе используем feat слой на актуальную дату. На трейне – в прошлом

Если мы считаем фичи на каждый день, то здесь есть отставание фичей внутри одного дня. То есть гранулярность расчета datamart немного ограничивает его возможности

Можно сделать на связки s3 + parquet + polars, но это не самая эффективная связка. Однако она параллелится по датам и по фичам

Также для больших окон по времени может потребоваться существенное число оперативной памяти

Recent inference в датамарт

- feat

- events

- user

- 2023-01-01.pq
 - 2023-02-02.pq

- item

- user-item

- user-category

- user-author

- cluster-item

- cluster-category

- orders

- item

- user-item

- user-category

- user-author

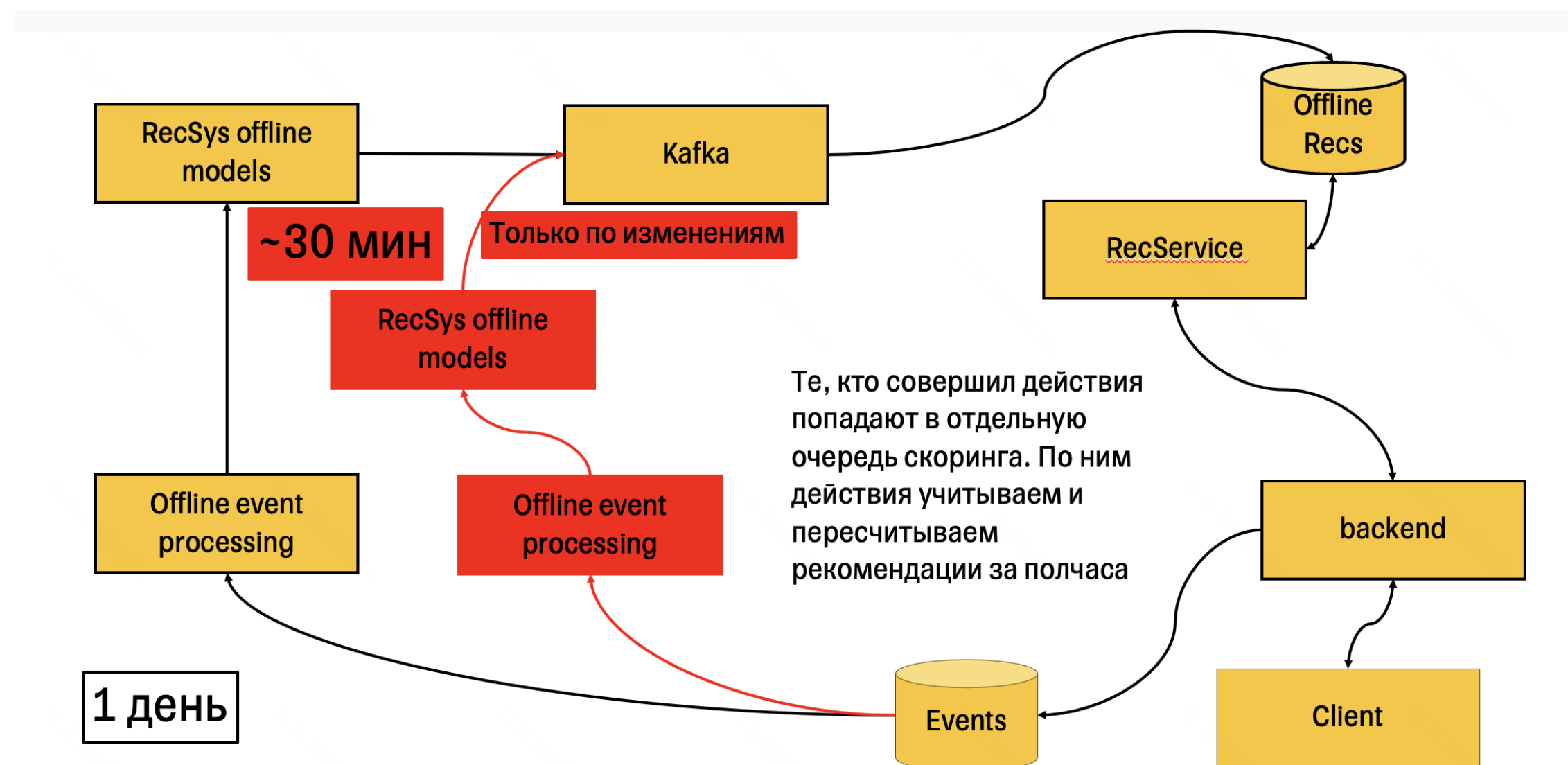
- cluster-item

- cluster-category

1) Пересчитать фичи за сегодня в feat слой каждые X минут

2) Для всех пользователей, кто обновился, пересчитать кандидатов и рекомендации, пересобрав basis

3) Отправить обновление по новым рекомендациям



Подход к построению №2 – Профиль пользователя



$y_n = y_{n-1} * 2^{t_{n-1}-t_n} + x_n$ y – значение фичи, x_n – значение инкремента в момент n

$$y_{n+1} = y_n * 2^{t_n-t_{n+1}} + x_{n+1}$$

$$y_{n+1} = (y_{n-1} * 2^{t_{n-1}-t_n} + x_n) * 2^{t_n-t_{n+1}} + x_{n+1}$$

$$y_{n+1} = y_{n-1} * 2^{t_{n-1}-t_n+t_n-t_{n+1}} + x_n * 2^{t_n-t_{n+1}} + x_{n+1}$$

$$y_{n+1} = y_{n-1} * 2^{t_{n-1}-t_{n+1}} + x_n * 2^{t_n-t_{n+1}} + x_{n+1}$$

$$y_{n+1} = x_{n+1} + x_n * 2^{t_n-t_{n+1}} + y_{n-1} * 2^{t_{n-1}-t_{n+1}}$$

Подход к построению №2 – Профиль пользователя



$$y_n = y_{n-1} * 2^{(t_{n-1}-t_n)/\alpha} + x_n$$

Тогда нам нужно:

- 1) Значение фичи на прошлый момент времени y_{n-1}
- 2) Прошлый момент времени t_{n-1}
- 3) Нужный момент времени t_n
- 4) Текущий инкремент x_n

В отличие от datamart:

- 1) Нам не нужно делать join фичей в прошлое, чтобы насчитать фичу
- 2) Нам можно все хранить в key-value хранилище в простом случае
- 3) Но нам надо подбирать α

Подход к построению №2 – Профиль пользователя



$$y_n = y_{n-1} * 2^{(t_{n-1}-t_n)/\alpha} + x_n$$

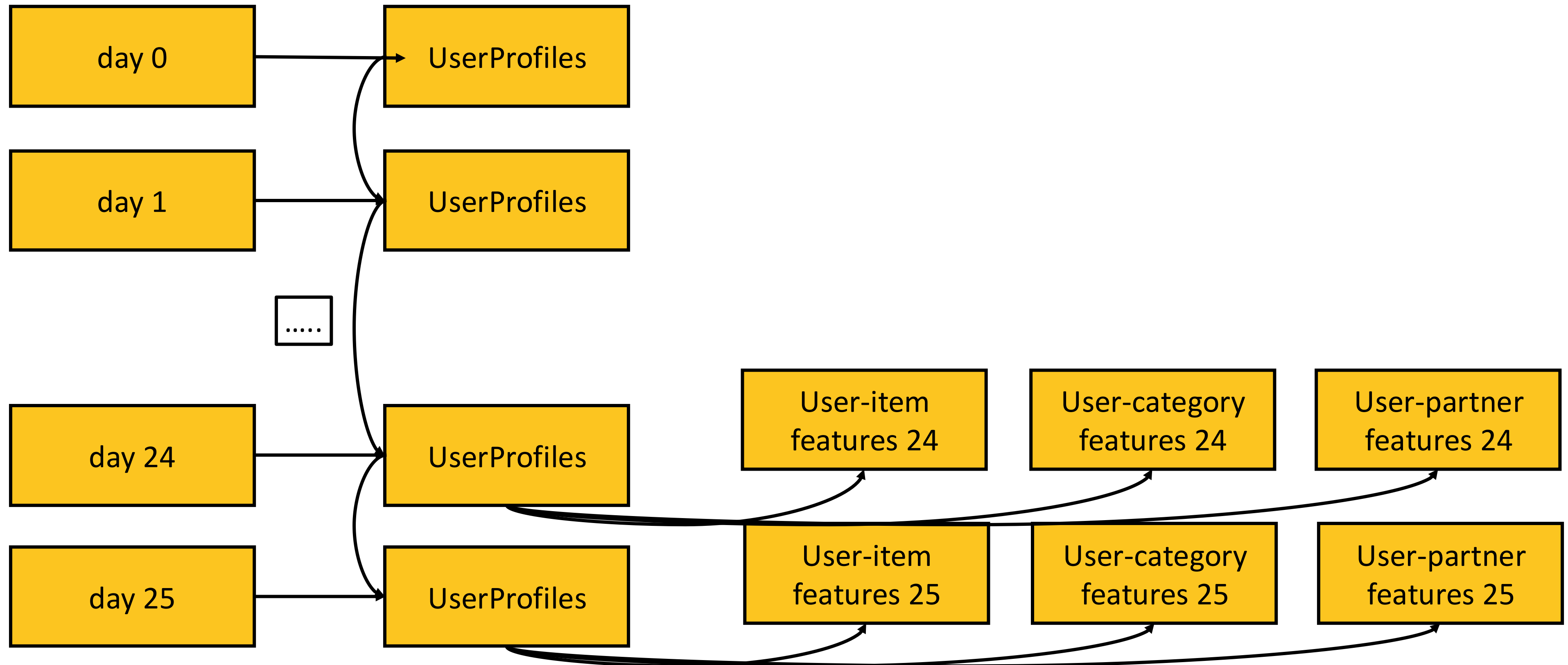
Создадим «профиль пользователя»:

- 1) Хранит в себе актуальные готовые значения признаков user, user-item, user-category, user-author
- 2) Умеет обновлять состояние по одной строке данных взаимодействий
- 3) Умеет сохранять состояние в parquet

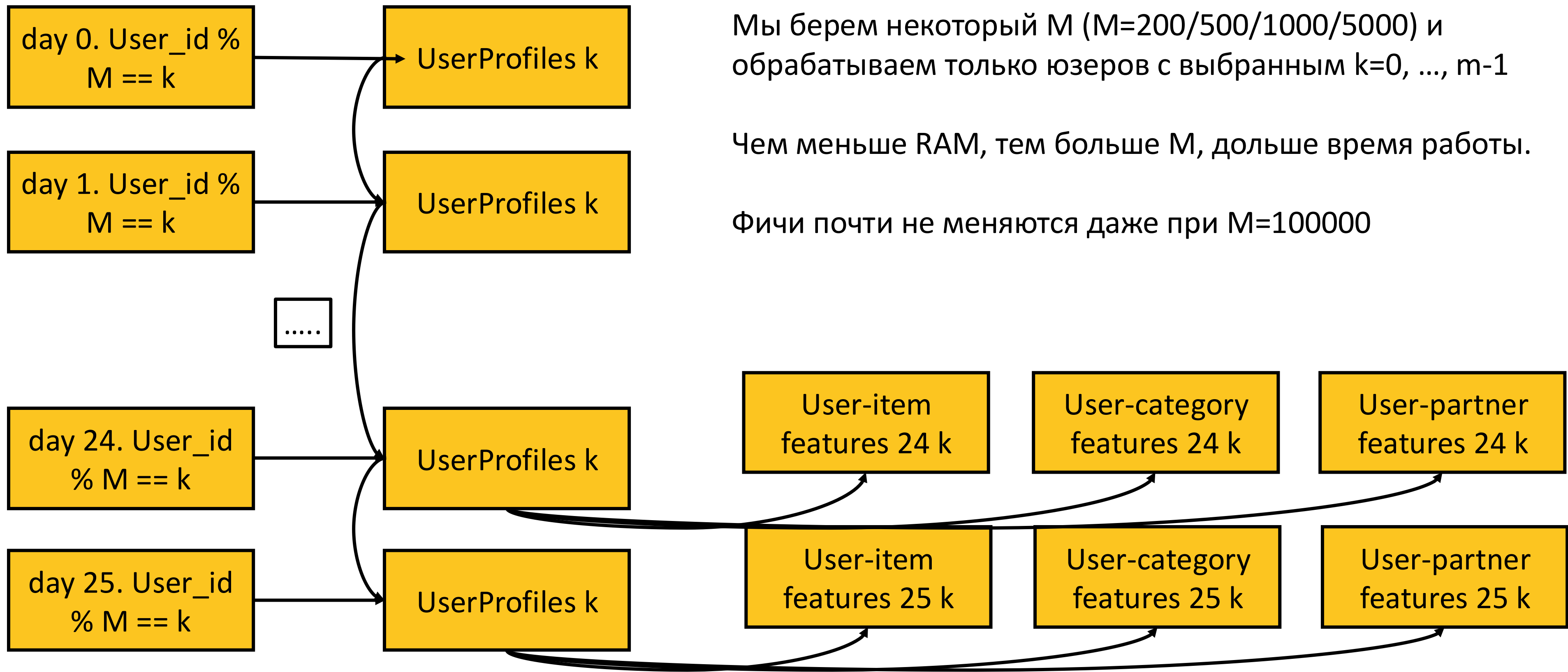
В нем:

- 1) Хранится словарь фичей по юзеру. Некоторые фичи обновляются экспоненциально, некоторые нет
- 2) В юзере есть словарь таких же фичей в разрезе category/author/partner, что делает фичи user-category, user-author

Подход к построению №2 – Профиль пользователя



Профиль пользователя - шардирование

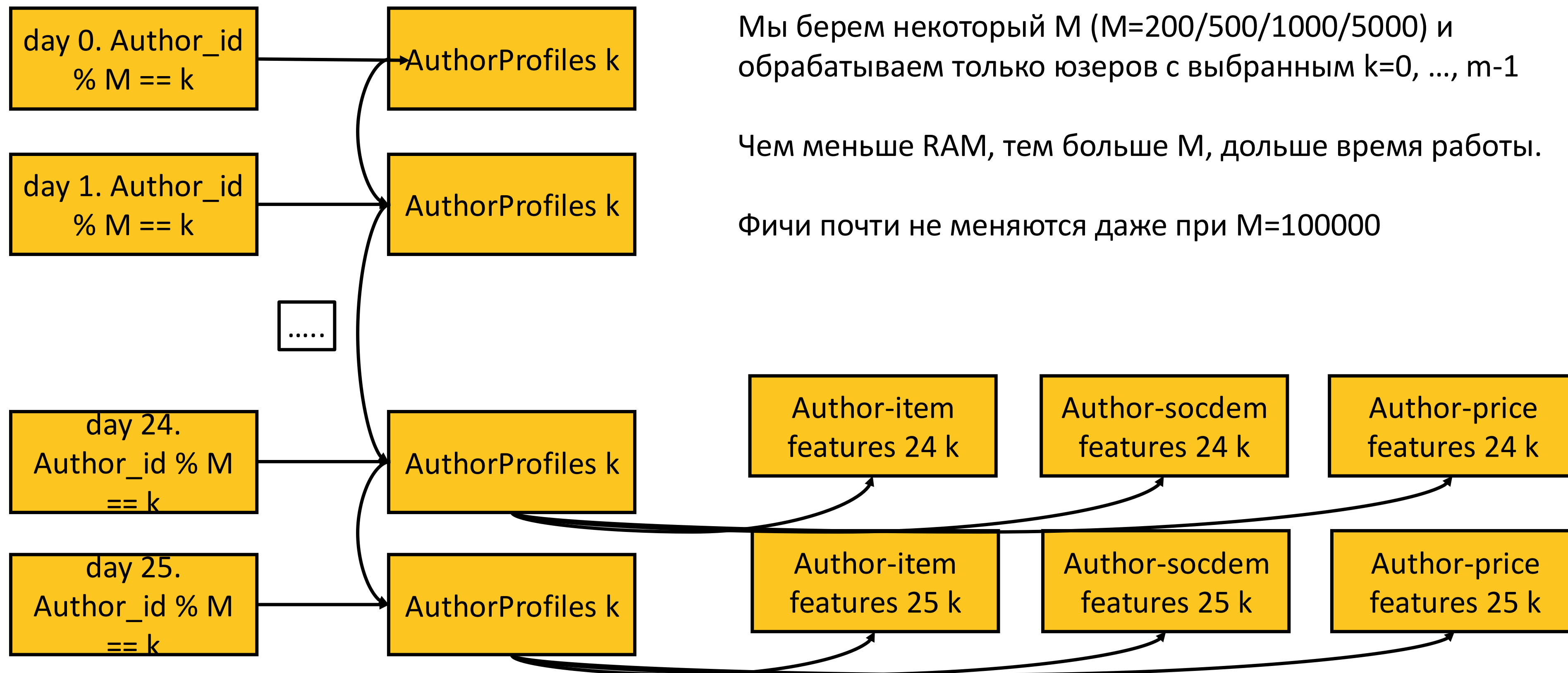


Мы берем некоторый M (M=200/500/1000/5000) и обрабатываем только юзеров с выбранным k=0, ..., m-1

Чем меньше RAM, тем больше M, дольше время работы.

Фичи почти не меняются даже при M=100000

Профиль чего угодно



Профиль чего угодно

Можно

Пересчитать профиль на
любую дату

Вы ретроспективно можете
считать фичи, передавая
правильный поток
исторических данных

Обновить профиль в
онлайне, то есть на основе
свежих событий успевать
обновлять данные и
пересчитывать их

Сбор данных: две концепции

Можно

SQL восстановление данных:

Ретроспективно насчитываем
фичи на нужные нам промежутки
времени

Log and wait

- 1) Логируем все фичи на
момент вызова
рекомендаций
- 2) Собираем таргеты по
залогированным
фичам и обучаем
модель

Сбор данных: две концепции

	SQL восстановление	Log and wait
Плюсы:	- Можно восстановить то, что не залогировалось - Не надо ждать, пока достаточно времени фича будет логироваться - Проще в реализации технически	- Логирует самые актуальные фичи несложным образом - Пайплайн затягивание новых фичей может быть проще: реализовываешь, логируешь, включаешь в обучение
Минусы:	- Может быть сложно посчитать самые актуальные фичи на момент запроса - Некоторые фичи в целом невозможно будет восстановить	- Может быть не залогировано то, что надо - Нужно ждать, пока будет достаточно данных по новой фиче - Фича может не выстрелить и время+память будет потрачено на ожидание зря - Сложнее в реализации технически

Масштабирование по юзерам

Если у вас много клиентов:

- 1) Имеет смысл разбить их на бакеты и работать с каждым бакетом параллельно (например, разложить файлы по партициям)
- 2) Вы можете обучать модель только на X клиентах, но применять на $10X$ клиентах. Такие модели не должны зависеть от `user_id` (например, EASE)
- 3) Вы можете применять модели только на активных юзерах. Всегда можно выделить X самых активных и варьировать X
- 4) Вы можете делать полуперсональные рекомендации, и работать только для кластеров пользователей. Например, пол – возраст – гео – доход – девайс – семейное положение – образование – и считать рекомендации по ним

Масштабирование по айтемам

Если у вас много айтемов:

- 1) Может иметь смысл ограничить каталог по популярности. Например, у вас 100к айтемов, но вы ограничиваете свои рекомендации только на топ-5к айтемов, по которым больше всего данных. Вы уменьшаете recall, но улучшаете быстродействие
- 2) Вы можете делать рекомендации не по айтемам, а по категориям. Например, раньше в Авито рекомендации делались на основе категорий, а сами объявления внутри категорий выбирались по популярности. Объявлений может быть миллиарды, а категорий всего 5к.
- 3) Можно выстраивать многоуровневый пайплайн отбора кандидатов, стараясь не терять релевантных на разных этапах
- 4) Можно разделять айтемы логически, например рекомендации только новых товаров (это бизнес логика отбора кандидатов сразу срежет много айтемов). Например в соцсети могут браться только посты, опубликованные за последнюю неделю.
- 5) Используйте ANN по возможности для быстрого поиска векторов

Масштабирование по айтемам. Кейс EASE

USERS * ITEMS

1				1				
				1		1		
1		1					1	1
					1			
	1							
			1					1
					1	1		
1		1					1	

X

ITEMS * ITEMS

0	.	.		.	.	.	.	.
.	0	.	.	.	.	.	.	.
.	.	0	.	.	.	.	.	.
.	.	.	0	.	.	.	.	.
.	.	.	.	0	.	.	.	.
.	.	.	.	.	0	.	.	.
.	.	.	.	.	.	0	.	.
.	.	.	.	.	.	.	0	.
.	.	.	.	.	.	.	.	0

=

USERS * ITEMS

1	.	.	.	.	.	.	.	.
				1		1		
1		1					1	1
					1			
	1							
			1					1
					1	1		
1		1					1	

Масштабирование по айтемам. Кейс EASE

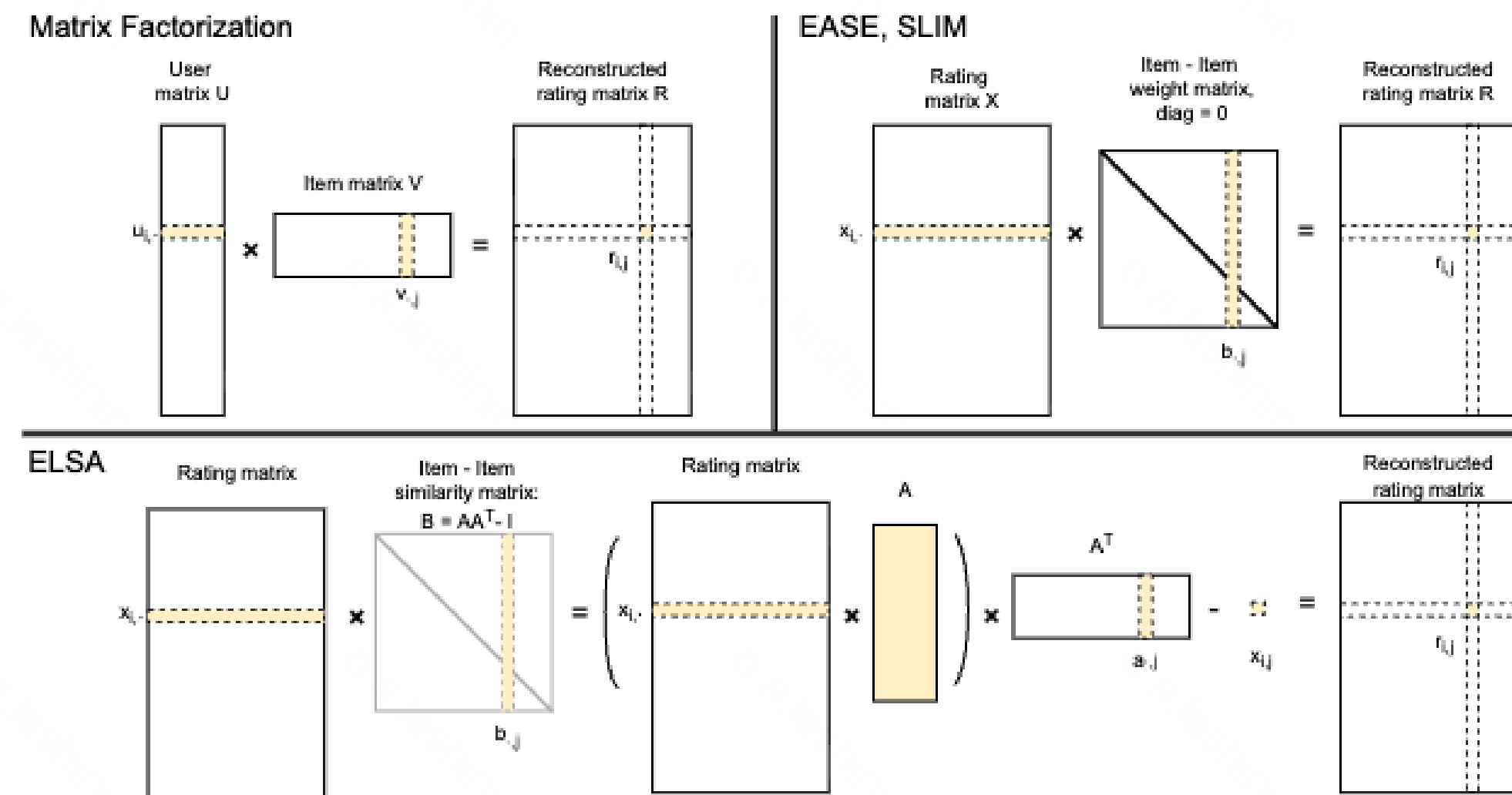


Figure 1: Differences between matrix factorization, traditional shallow autoencoders and ELSA.

Optimization problem: ELSA regards to the following optimization:

$$\min_A \|f(X) - f(X(AA^T - I))\|_F^2 \quad \text{s.t.} \quad \|A_{a,\cdot}\|^2 = 1 \quad (1)$$

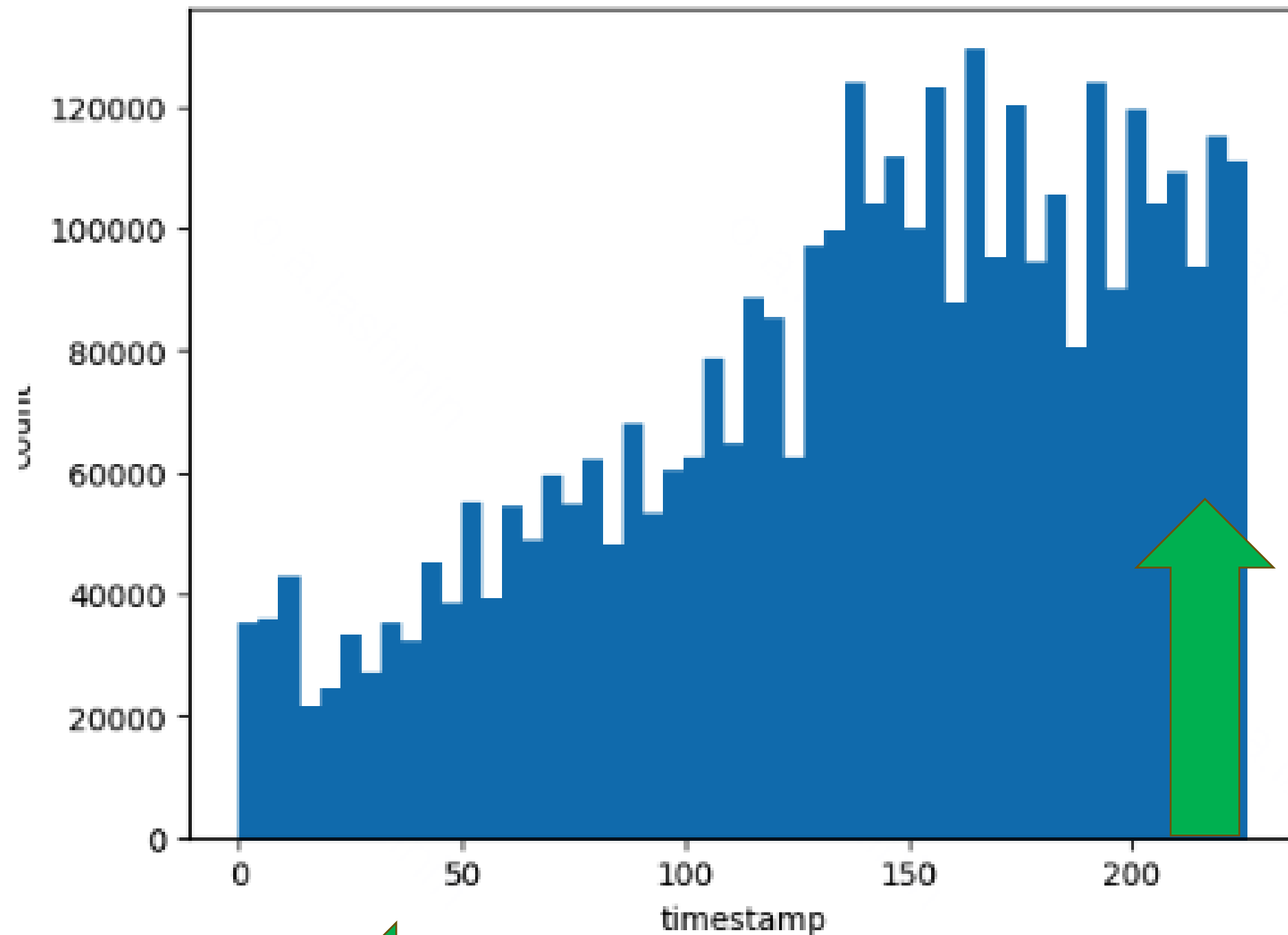
where $A \in \mathbb{R}^{I \times r}$ is a latent matrix, I is the identity matrix and $r \ll I$ is a (tunable) hyperparameter that reduces the latent space and helps to avoid over-fitting. ELSA draws inspiration from the cosine

Масштабирование по айтемам. Кейс EASE → ELSA

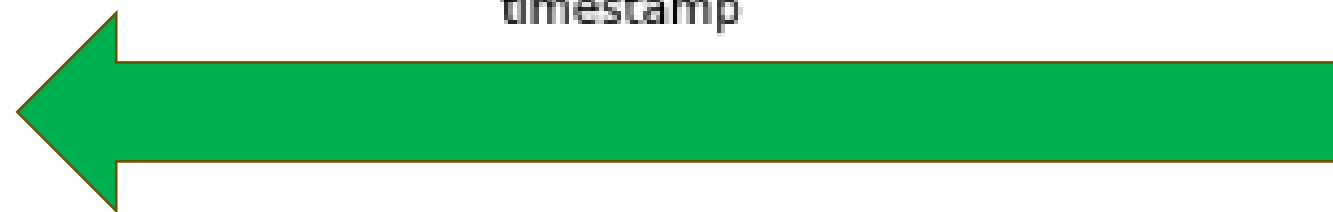
Table 4. Offline evaluation results. Each value represents the mean and standard deviation obtained from 10 independent runs. Peak memory was measured with Scalene [3].

Dataset	Model	Recall@20	Recall@50	NDCG@100	Coverage@20	Time [s]	Peak Memory (+GPU) [GiB]
GB-10k	KNN	0.2290 ± 0.0032	0.3431 ± 0.0031	0.3550 ± 0.0038	0.3797 ± 0.0047	-	1.9
	ALS MF	0.2360 ± 0.0027	0.3442 ± 0.0032	0.3606 ± 0.0029	0.1492 ± 0.0029	59.26	1.1
	EASE	0.3366 ± 0.0032	0.4834 ± 0.0033	0.4824 ± 0.0034	0.4856 ± 0.0057	8.03	3.9
	SANSA	0.3110 ± 0.0036	0.4542 ± 0.0036	0.4564 ± 0.0042	0.6750 ± 0.0049	88.59	2.4
	ELSA	0.3503 ± 0.0034	0.4898 ± 0.0033	0.4897 ± 0.0038	0.8070 ± 0.0052	40.34	1.4 (+2.2)
	Sparse ELSA	0.3502 ± 0.0035	0.4894 ± 0.0031	0.4892 ± 0.0039	0.8137 ± 0.0050	15.11	1.4 (+4.8)
ML-20M	KNN	0.3593 ± 0.0024	0.4882 ± 0.0030	0.3961 ± 0.0018	0.1884 ± 0.0031	-	4.8
	ALS MF	0.3567 ± 0.0015	0.4957 ± 0.0025	0.3948 ± 0.0015	0.1425 ± 0.0033	273.48	3
	EASE	0.3867 ± 0.0017	0.5158 ± 0.0024	0.4253 ± 0.0017	0.2839 ± 0.0049	40.35	14.2
	SANSA	0.3342 ± 0.0017	0.4657 ± 0.0022	0.3734 ± 0.0017	0.2568 ± 0.0074	110.05	7.5
	ELSA	0.3926 ± 0.0017	0.5277 ± 0.0026	0.4283 ± 0.0018	0.5555 ± 0.0075	163.39	2.3 (+1.9)
	Sparse ELSA	0.3928 ± 0.0018	0.5281 ± 0.0026	0.4286 ± 0.0019	0.5409 ± 0.0091	30.57	2.8 (+3.0)
Netflix	KNN	0.2979 ± 0.0010	0.3787 ± 0.0009	0.3368 ± 0.0008	0.1947 ± 0.0027	-	15.2
	ALS MF	0.3149 ± 0.0009	0.4068 ± 0.0009	0.3586 ± 0.0007	0.1493 ± 0.0015	1077.58	15.2
	EASE	0.3535 ± 0.0007	0.4406 ± 0.0007	0.3948 ± 0.0007	0.3936 ± 0.0034	90.15	19.6
	SANSA	0.3044 ± 0.0010	0.3924 ± 0.0011	0.3472 ± 0.0008	0.4257 ± 0.0058	2234.87	16.1
	ELSA	0.3527 ± 0.0009	0.4408 ± 0.0007	0.3940 ± 0.0007	0.7247 ± 0.0078	390.64	15.2 (+3.6)
	Sparse ELSA	0.3528 ± 0.0009	0.4409 ± 0.0007	0.3941 ± 0.0007	0.7221 ± 0.0083	140.03	15.2 (+6.0)
MCD	KNN	0.2265 ± 0.0021	0.3012 ± 0.0018	0.2845 ± 0.0019	0.6330 ± 0.0026	-	6.9
	ALS MF	0.2170 ± 0.0014	0.3017 ± 0.0016	0.2680 ± 0.0010	0.3121 ± 0.0017	1048.76	9.7
	EASE	0.3326 ± 0.0026	0.4300 ± 0.0023	0.3912 ± 0.0024	0.7869 ± 0.0022	222.03	62.1

Масштабирование по данным

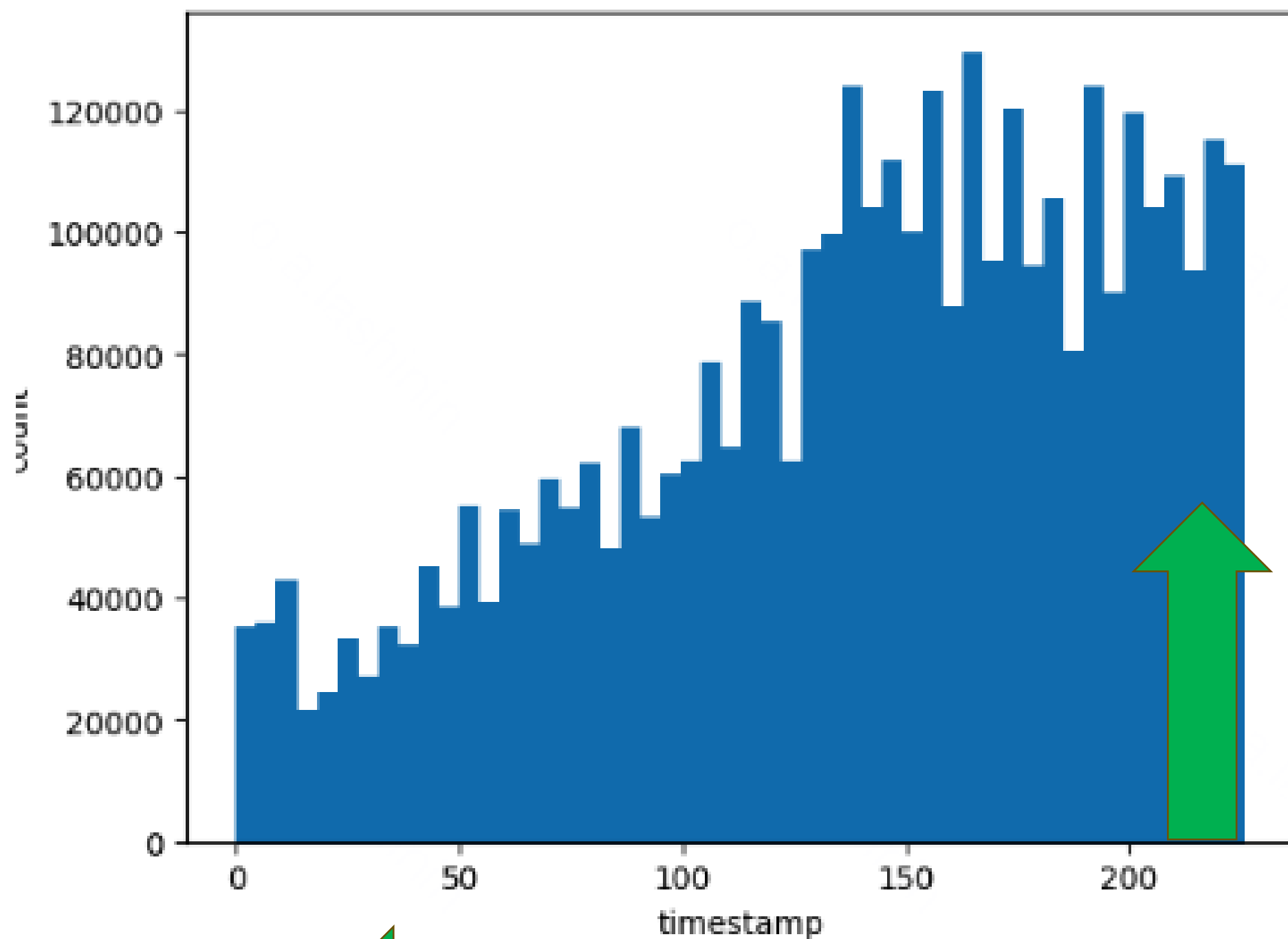


Можем изначально использовать не все данные и просто добавлять их (вширь). Здесь все ок, надо делать, пока есть прирост по качеству



Можем расширяться влево по данным (вглубь)

Масштабирование по данным по времени



Мы углубляемся по данным либо по X – фичам, либо по Y – таргетам.

По X – это скорее хорошо, но чем дальше мы идем, тем более старые данные мы забираем. Важно, чтобы ваши фичи были устойчивы к старым данным!

По Y – чем дальше идем, тем менее актуальные паттерны собираем. Либо берем айтемы, которых уже нет; либо смешиваем сезонности (обучаемся на зиме, хотя сейчас уже лето).

Поэтому масштабирование по данным вглубь по датам надо делать очень аккуратно!

Можем расширяться влево по данным (вглубь)

Спасибо за
внимание